NAAN MUDHALVAN

Project Report

Implement Client Script & UI Policy (Incident)

ServiceNow micro project

Team ID	SWTID-2026-4524
Team Size	4
Team Leader	Swathi R
Team Member	Sandhiya D
Team Member	Sandhya Charlet Calista S
Team Member	Mallemkonda Dallyjasmine

1. INTRODUCTION

1.1 Project Overview

Incident records need consistent and accurate data for effective triage, routing and resolution. This project, Implement Client Script & UI Policy (Incident), uses ServiceNow client-side controls to enforce data integrity on the Incident form. It combines a UI Policy with three Client Scripts so that fields respond to the Impact value, values are filled in automatically and records that break the business rules are stopped before they are saved.

The solution is made of the following parts:

- **UI Policy "High Impact Control":** applies when Impact is 1 - High and makes the Assignment group mandatory, with Reverse if false enabled.
- **UI Policy Action:** makes the Urgency field read-only while the policy condition is met.
- **onChange Client Script:** sets Urgency to High and shows an info message when Impact changes to High.
- **onSubmit Client Script:** blocks the save and shows an error on Assigned To when Impact is High and Assigned To is empty.
- **onCellEdit Client Script:** blocks changes to the State field when it is edited directly in the list view.

1.2 Purpose

The purpose of the project is to demonstrate how UI Policies and Client Scripts can work together to enforce data integrity on Incident records. The solution aims to:

- make fields mandatory, read-only or auto-populated based on the Impact value;
- prevent Incident records from being submitted when required conditions are not met;
- improve data quality, reporting accuracy and SLA compliance across the platform;
- provide a lightweight, easy-to-implement example of good practice for form usability.

2. IDEATION PHASE

2.1 Problem Statement

Incident records often require consistent and accurate data entry to ensure effective triage, routing and resolution. However, relying solely on user awareness and manual checks can lead to incomplete, inconsistent or incorrect data being submitted. This can impact reporting accuracy, SLA compliance and overall service quality.

To address this challenge, conditional field behaviour and validation need to be enforced directly at the user interface level. Standardized UI Policies and Client Scripts control field visibility, mandatory status and save-time validation, so that Incident records meet the defined business rules before they are submitted.

2.2 Empathy Map Canvas

The empathy map below is drawn for the main user of the Incident form, an agent who creates and updates Incident records.

Persona	Incident user (agent / caller) who creates and updates Incident records on the Incident form and list
Says	"Which fields do I have to fill for a high impact Incident?" "I need to log this quickly."
Thinks	"I hope I have not missed a required field." "Will this Incident be routed and resolved on time?"
Does	Selects Impact and Urgency, fills Assigned To, saves the Incident, and sometimes edits State directly from the list
Feels	Unsure about the rules, frustrated when a record comes back incomplete, reassured when the form guides them
Pains	Missing or wrong data, manual checks, accidental State changes made in the list view
Gains	Clear messages, Urgency set automatically, consistent records and better SLA reporting

2.3 Brainstorming

The team gathered, selected the problem statement above and listed ideas, which were then grouped by theme and prioritised on importance and feasibility.

Group	Idea	What it does
UI Policy	High Impact Control (Incident)	Applies extra controls when Impact is 1 - High; Assignment group mandatory; Reverse if false = true
UI Policy	UI Policy Action on Urgency	Makes Urgency read-only when Impact is High
Client Script	onChange: Auto set urgency for high impact	Sets Urgency to High and shows an info message when Impact changes to High
Client Script	onSubmit: Prevent save if Assigned To missing	Blocks the save and shows an error on Assigned To for High impact Incidents
Client Script	onCellEdit: Prevent state change via list edit	Shows an alert and blocks State changes made directly in the list
Testing	Test every scenario	Confirms all rules work and revert correctly when Impact is no longer High

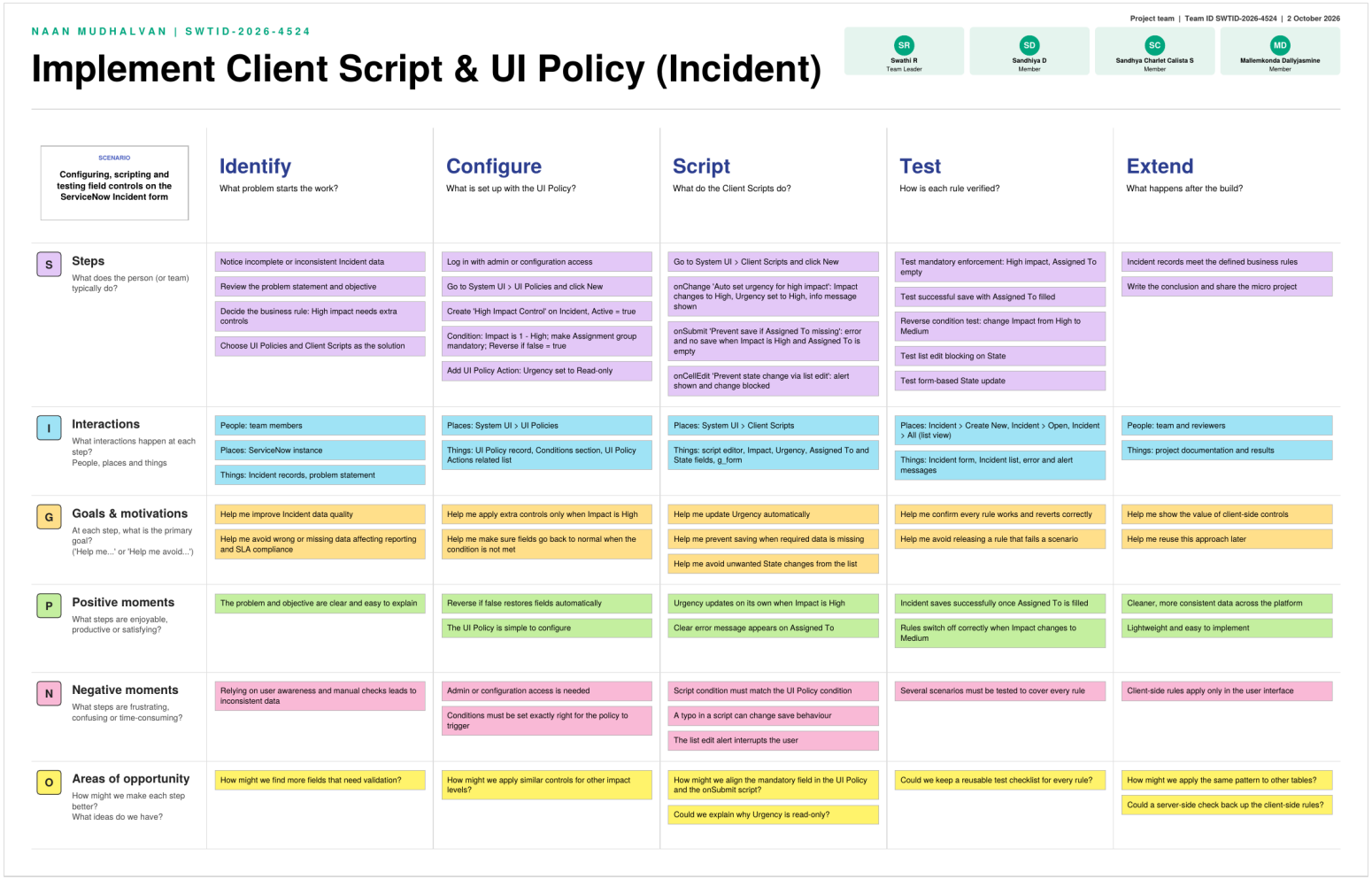
Prioritisation of the ideas:

Idea	Importance	Feasibility	Priority
High Impact Control UI Policy (with Urgency read-only action)	High	High	Implement first
onSubmit: Prevent save if Assigned To missing	High	High	Implement first
onChange: Auto set urgency for high impact	High	High	Implement next
onCellEdit: Prevent state change via list edit	Medium	High	Implement next
Testing all scenarios	High	High	Final validation

3. REQUIREMENT ANALYSIS

3.1 Customer Journey Map

The journey map follows the ServiceNow administrator or developer who builds and verifies the Incident form controls, across five phases: Identify, Configure, Script, Test and Extend.



NAAN MUDHALVAN | SWTID-2026-4524

Implement Client Script & UI Policy (Incident)

Project team | Team ID SWTID-2026-4524 | 2 October 2026

Swathi R (SR)	Sandhiya D (SD)	Sandhya Charlet Calista S (SC)	Mallemkonda Dallyjasmine (MD)
Team Leader	Member	Member	Member

SCENARIO

Configuring, scripting and testing field controls on the ServiceNow Incident form

	Identify What problem starts the work?	Configure What is set up with the UI Policy?	Script What do the Client Scripts do?	Test How is each rule verified?	Extend What happens after the build?
S Steps What does the person (or team) typically do?	Notice incomplete or inconsistent Incident data Review the problem statement and objective Decide the business rule: High impact needs extra controls Choose UI Policies and Client Scripts as the solution	Log in with admin or configuration access Go to System UI > UI Policies and click New Create 'High Impact Control' on Incident, Active = true Condition: Impact is 1 - High; make Assignment group mandatory; Reverse if false = true Add UI Policy Action: Urgency set to Read-only	Go to System UI > Client Scripts and click New onChange 'Auto set urgency for high impact': Impact changes to High, Urgency set to High, info message shown onSubmit 'Prevent save if Assigned To missing': error and no save when Impact is High and Assigned To is empty onCellEdit 'Prevent state change via list edit': alert shown and change blocked	Test mandatory enforcement: High impact, Assigned To empty Test successful save with Assigned To filled Reverse condition test: change Impact from High to Medium Test list edit blocking on State Test form-based State update	Incident records meet the defined business rules Write the conclusion and share the micro project
I Interactions What interactions happen at each step? People, places and things	People: team members Places: ServiceNow instance Things: Incident records, problem statement	Places: System UI > UI Policies Things: UI Policy record, Conditions section, UI Policy Actions related list	Places: System UI > Client Scripts Things: script editor, Impact, Urgency, Assigned To and State fields, g_form	Places: Incident > Create New, Incident > Open, Incident > All (list view) Things: Incident form, Incident list, error and alert messages	People: team and reviewers Things: project documentation and results
G Goals & motivations At each step, what is the primary goal? ('Help me...' or 'Help me avoid...')	Help me improve Incident data quality Help me avoid wrong or missing data affecting reporting and SLA compliance	Help me apply extra controls only when Impact is High Help me make sure fields go back to normal when the condition is not met	Help me update Urgency automatically Help me prevent saving when required data is missing Help me avoid unwanted State changes from the list	Help me confirm every rule works and reverts correctly Help me avoid releasing a rule that fails a scenario	Help me show the value of client-side controls Help me reuse this approach later
P Positive moments What steps are enjoyable, productive or satisfying?	The problem and objective are clear and easy to explain	Reverse if false restores fields automatically The UI Policy is simple to configure	Urgency updates on its own when Impact is High Clear error message appears on Assigned To	Incident saves successfully once Assigned To is filled Rules switch off correctly when Impact changes to Medium	Cleaner, more consistent data across the platform Lightweight and easy to implement
N Negative moments What steps are frustrating, confusing or time-consuming?	Relying on user awareness and manual checks leads to inconsistent data	Admin or configuration access is needed Conditions must be set exactly right for the policy to trigger	Script condition must match the UI Policy condition A typo in a script can change save behaviour The list edit alert interrupts the user	Several scenarios must be tested to cover every rule	Client-side rules apply only in the user interface
O Areas of opportunity How might we make each step better? What ideas do we have?	How might we find more fields that need validation?	How might we apply similar controls for other impact levels?	How might we align the mandatory field in the UI Policy and the onSubmit script? Could we explain why Urgency is read-only?	Could we keep a reusable test checklist for every rule?	How might we apply the same pattern to other tables? Could a server-side check back up the client-side rules?

Figure 3.1: Customer journey map for the Incident form controls

3.2 Solution Requirement

Functional requirements

ID	Requirement	Description
FR-1	High Impact Control UI Policy	A UI Policy on the Incident table, active, with the condition Impact is 1 - High
FR-2	Mandatory Assignment group	Assignment group becomes mandatory when Impact is High; Reverse if false = true restores it when the condition is not met
FR-3	Urgency read-only	A UI Policy Action makes Urgency read-only when Impact is High and editable otherwise
FR-4	Auto set Urgency	An onChange Client Script on Impact sets Urgency to High and shows an info message
FR-5	Save validation	An onSubmit Client Script blocks the save and shows an error on Assigned To when Impact is High and Assigned To is empty
FR-6	List edit control	An onCellEdit Client Script on State shows an alert and blocks State changes made in the list view
FR-7	Form update allowed	State can still be changed and saved from the Incident form

Non-functional requirements

Requirement	Description
Data quality	Incidents are created with complete and valid information, keeping data consistent across the platform
Usability	Clear info, error and alert messages tell the user what to do; fields change behaviour automatically
Maintainability	The rules are simple, lightweight and easy to configure in System UI, and can be reused for other fields
Performance	The controls are client-side rules that respond immediately in the browser when a field changes or a record is saved
Access	Creating the UI Policy and Client Scripts needs administrative or configuration access to the ServiceNow instance

3.3 Data Flow Diagram

The Level 0 data flow diagram shows how the Administrator configures the control rules, how those rules are applied when the Incident User works on the form and list, and how valid records reach the Incident table.

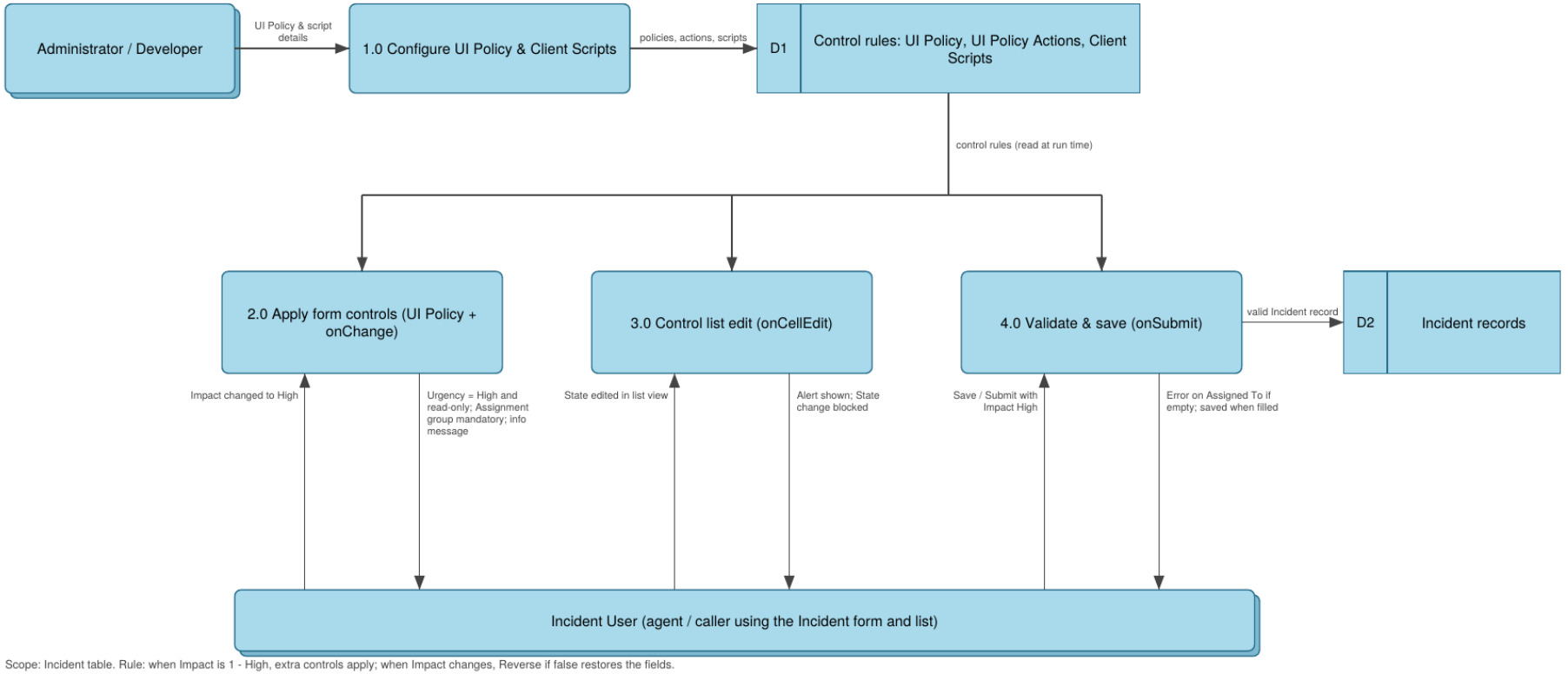


Figure 3.3: Level 0 data flow diagram

3.4 Technology Stack

Layer	Technology
Platform	ServiceNow instance (web interface, Incident application)
Data	Incident table and Incident records
Configuration	System UI > UI Policies, UI Policy Actions and Client Scripts
Scripting	JavaScript client scripts using the g_form API (onChange, onSubmit, onCellEdit)
Access	Web browser; administrative or configuration access to the instance

4. PROJECT DESIGN

4.1 Problem Solution Fit

Problem	How the solution fits
Incident data is incomplete or inconsistent because it depends on user awareness and manual checks	Rules are enforced on the form itself, so users cannot skip required information for high impact Incidents
Wrong or missing data affects reporting accuracy and SLA compliance	Required fields and automatic values keep records consistent and complete before they are saved
Fields should behave differently only when Impact is High	The UI Policy condition and Reverse if false apply the controls only when needed and restore normal behaviour otherwise
State can be changed accidentally from the list view	The onCellEdit script blocks list edits of State and asks the user to open the Incident

4.2 Proposed Solution

The proposed solution uses ServiceNow client-side controls on the Incident form and list. When Impact is set to High, the High Impact Control UI Policy makes Assignment group mandatory and its action makes Urgency read-only, while the onChange script sets Urgency to High and shows an info message. When the user saves, the onSubmit script checks that Assigned To is filled for High impact Incidents. In the list view, the onCellEdit script stops direct changes to State. If Impact is changed to a different value, Reverse if false restores the fields to their normal behaviour.

Component	Purpose
UI Policy: High Impact Control	Triggers on Impact is 1 - High; Assignment group mandatory; Reverse if false
UI Policy Action: Urgency	Read-only Urgency while the condition is met
onChange: Auto set urgency for high impact	Urgency = High and info message when Impact becomes High
onSubmit: Prevent save if Assigned To missing	Error and no save when Impact is High and Assigned To is empty
onCellEdit: Prevent state change via list edit	Alert and no change when State is edited in the list

4.3 Solution Architecture

The Administrator creates the configuration records in System UI. These rules load in the browser when an Incident form or list is opened. The UI Policy, onChange, onCellEdit and onSubmit controls respond to the Incident User, and a valid record is stored in the Incident table.

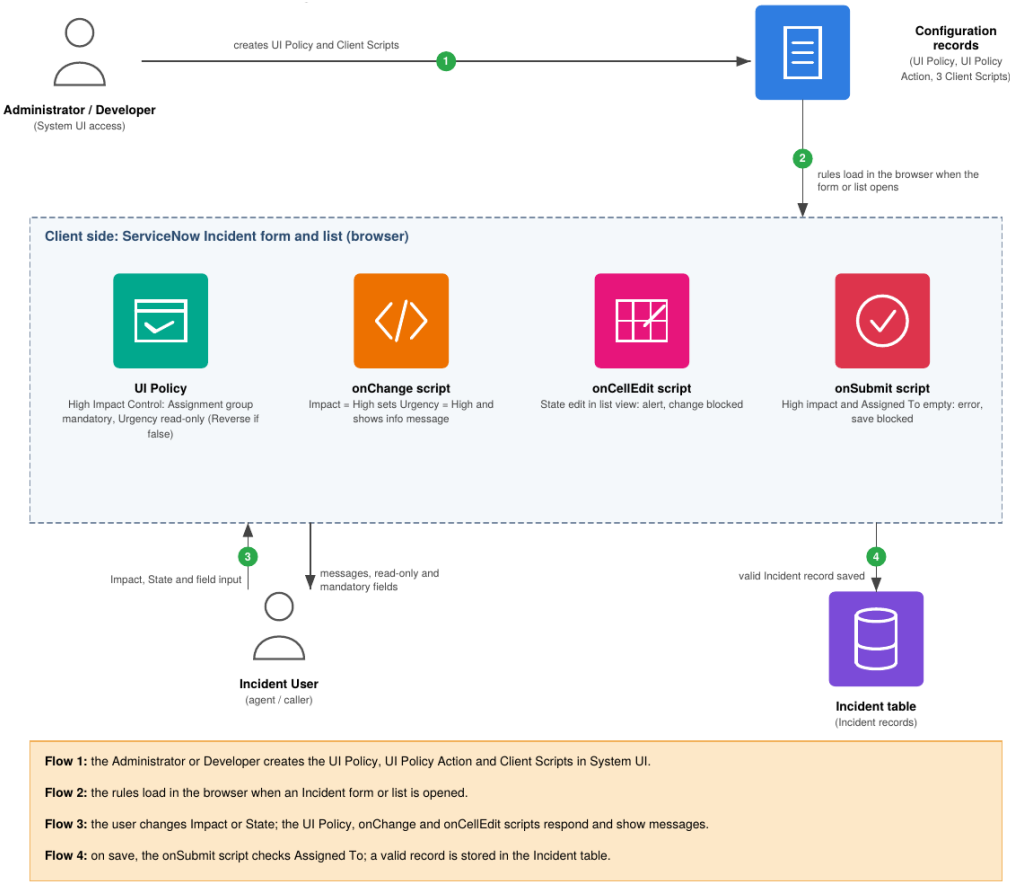


Figure 4.3: Solution architecture and data flow

5. PROJECT PLANNING & SCHEDULING

5.1 Project Planning

Product backlog, sprint schedule and estimation

Sprint	Epic	Story No.	User Story / Task	Points	Priority	Team
Sprint-1	UI Policy	USN-1	As an administrator, I can create a UI Policy named High Impact Control on the Incident table with the condition Impact is 1 - High.	3	High	All members
Sprint-1		USN-2	As an administrator, I can make Assignment group mandatory in the UI Policy and set Reverse if false to true.	3	High	All members
Sprint-1		USN-3	As an administrator, I can add a UI Policy Action that makes the Urgency field read-only.	2	High	All members
Sprint-2	Client Scripts	USN-4	As an administrator, I can create an onChange Client Script on the Impact field to set Urgency automatically.	3	High	All members
Sprint-2		USN-5	As an administrator, I can create an onSubmit Client Script that prevents saving when Assigned To is missing.	3	High	All members
Sprint-2		USN-6	As an administrator, I can create an onCellEdit Client Script on the State field.	2	Medium	All members
Sprint-3	Testing	USN-7	As an administrator, I can test every rule, including the reverse condition, list edit blocking and form-based update.	3	Medium	All members
Sprint-2	Incident form	USN-8	As an incident user, I see Urgency set automatically when I select High impact.	2	High	All members
Sprint-2		USN-9	As an incident user, I cannot save a High impact Incident without Assigned To.	2	High	All members
Sprint-3		USN-10	As an incident user, I can still change State from the Incident form.	2	Medium	All members

Project tracker

Sprint	Total Story Points	Duration	Sprint Start Date	Sprint End Date (Planned)	Story Points Completed (as on Planned End Date)	Sprint Release Date (Actual)
Sprint-1	8	6 Days	5 Oct 2026	10 Oct 2026		
Sprint-2	12	6 Days	12 Oct 2026	17 Oct 2026		
Sprint-3	5	6 Days	19 Oct 2026	24 Oct 2026		

Velocity

The team plans 25 story points over 3 sprints of 6 days each, so the average velocity is 25 / 3 = 8.33 story points per sprint. The average velocity per iteration unit (story points per day) is AV = velocity / sprint duration = 8.33 / 6 = 1.39 story points per day.

Burndown chart

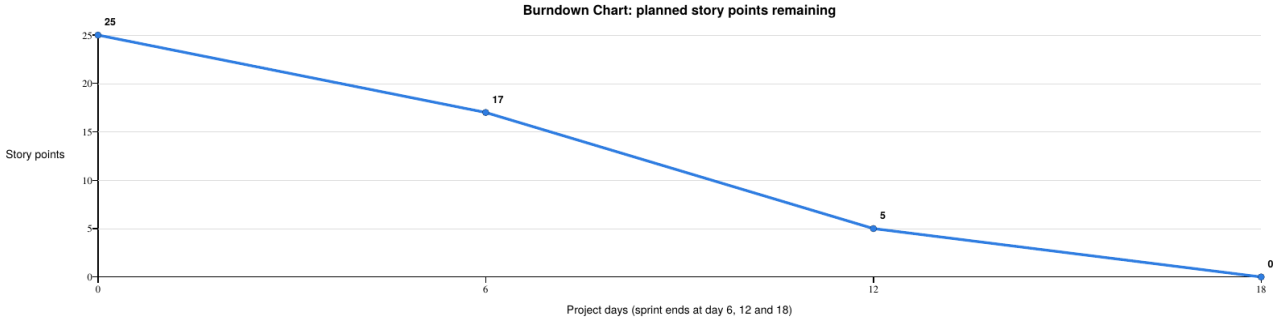


Figure 5.1: Planned burndown chart (story points remaining by project day)

6. FUNCTIONAL AND PERFORMANCE TESTING

6.1 Performance Testing

The configuration is verified with the five scenarios below. The UI Policy and Client Scripts are client-side rules, so each check runs in the browser at the moment a field changes or a record is saved.

No.	Test case	Steps	Expected result	Result
1	Mandatory enforcement	Create a new Incident, set Impact to High, leave Assigned To empty and click Submit	Incident is not saved; an error message says Assigned To is mandatory for High impact; Urgency is set to High and read-only	
2	Successful save	Fill Assigned To and click Submit	Incident saves successfully with no errors; Urgency, read-only and State rules still work	
3	Reverse condition	Open an Incident with Impact High and change Impact to Medium, then save	Assigned To is no longer mandatory, Urgency is editable and the record saves	
4	List edit blocking	In the Incident list, double-click State to edit it	An alert appears and State stays unchanged	
5	Form-based update	Open the Incident, change State on the form and click Update	State change is saved; other rules still apply	

The Result column is for the actual outcome of each test as observed on the instance.

7. RESULTS

7.1 Output Screenshots

The figures below are illustrative mock-ups of the configuration records and of the expected behaviour on the Incident form and list.

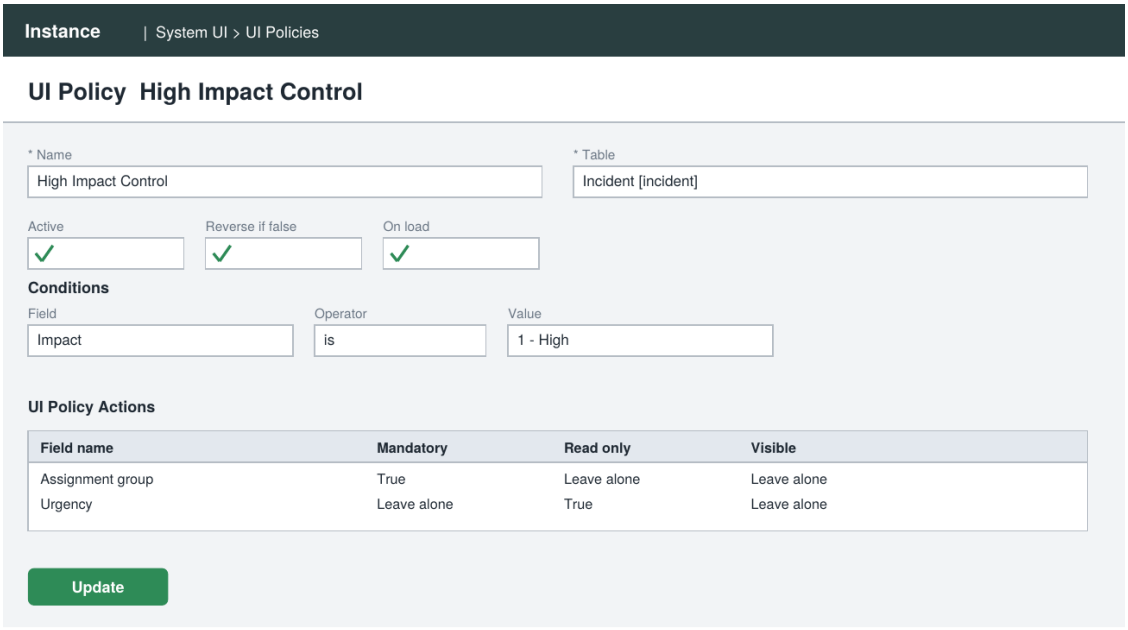


Figure 7.1: UI Policy "High Impact Control" with condition Impact is 1 - High (illustrative mock-up)

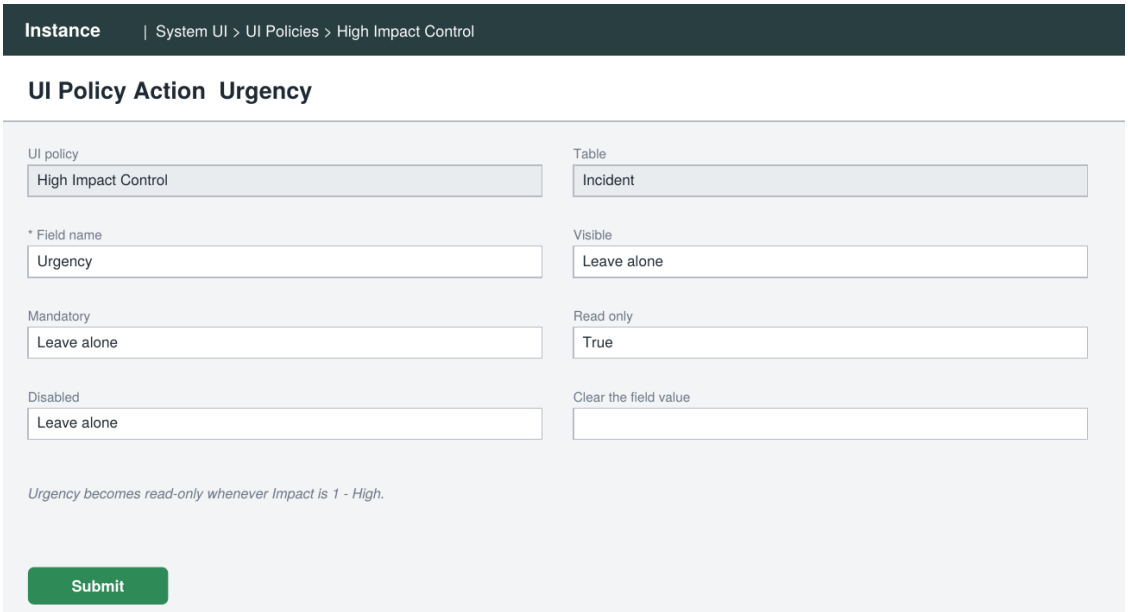


Figure 7.2: UI Policy Action: Urgency set to read-only (illustrative mock-up)

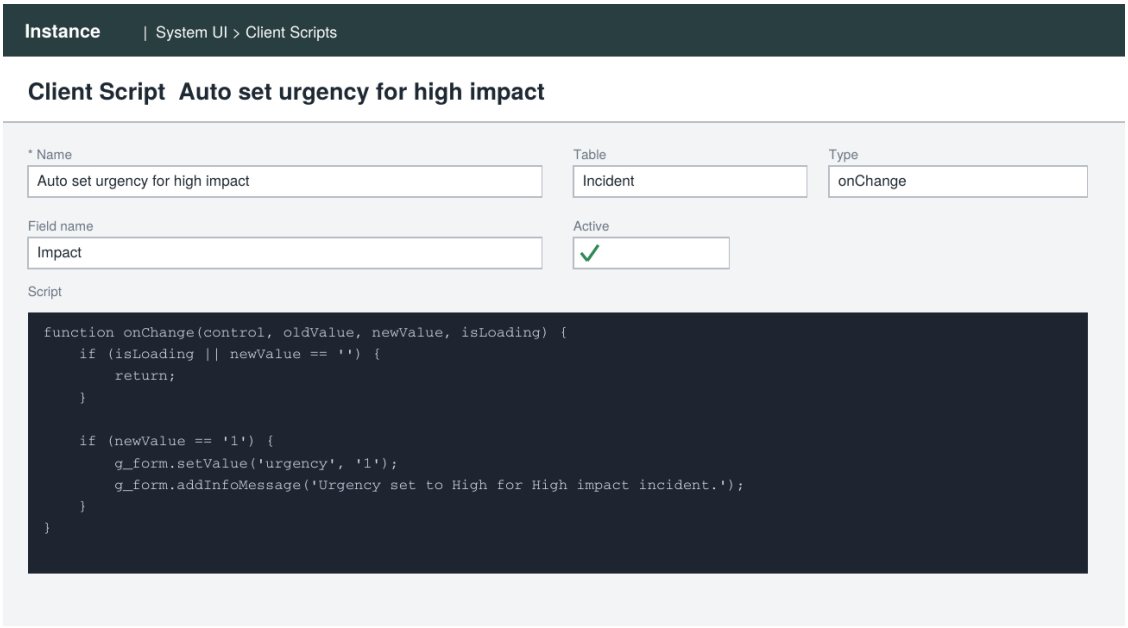


Figure 7.3: onChange Client Script: Auto set urgency for high impact (illustrative mock-up)

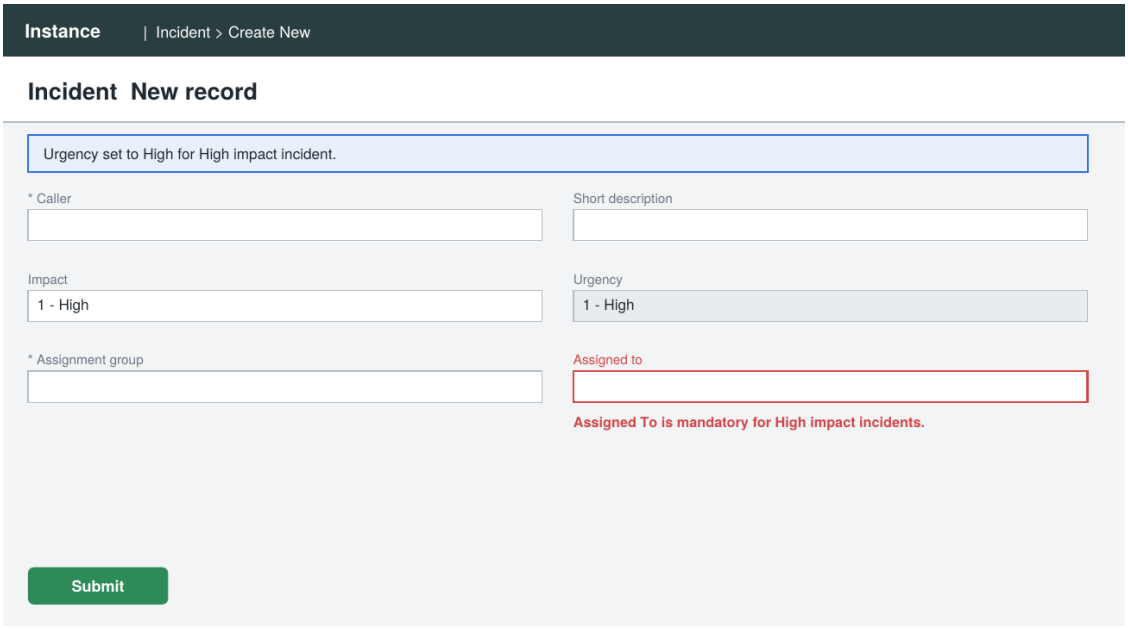


Figure 7.4: onSubmit Client Script: error shown when Assigned To is missing (illustrative mock-up)

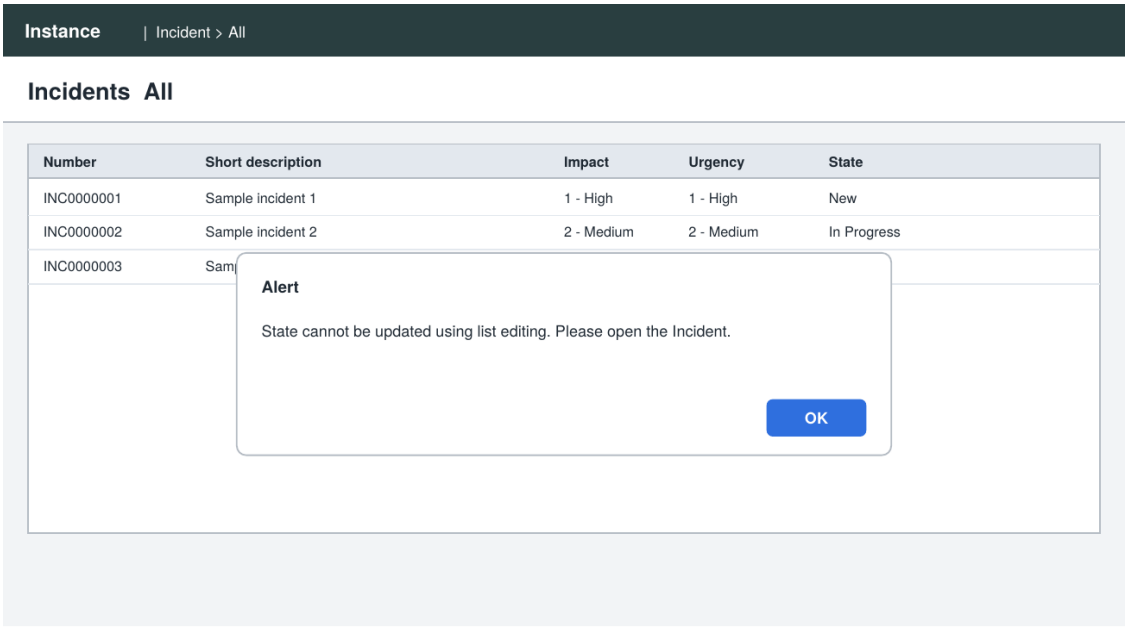


Figure 7.5: onCellEdit Client Script: alert shown on list edit of State (illustrative mock-up)

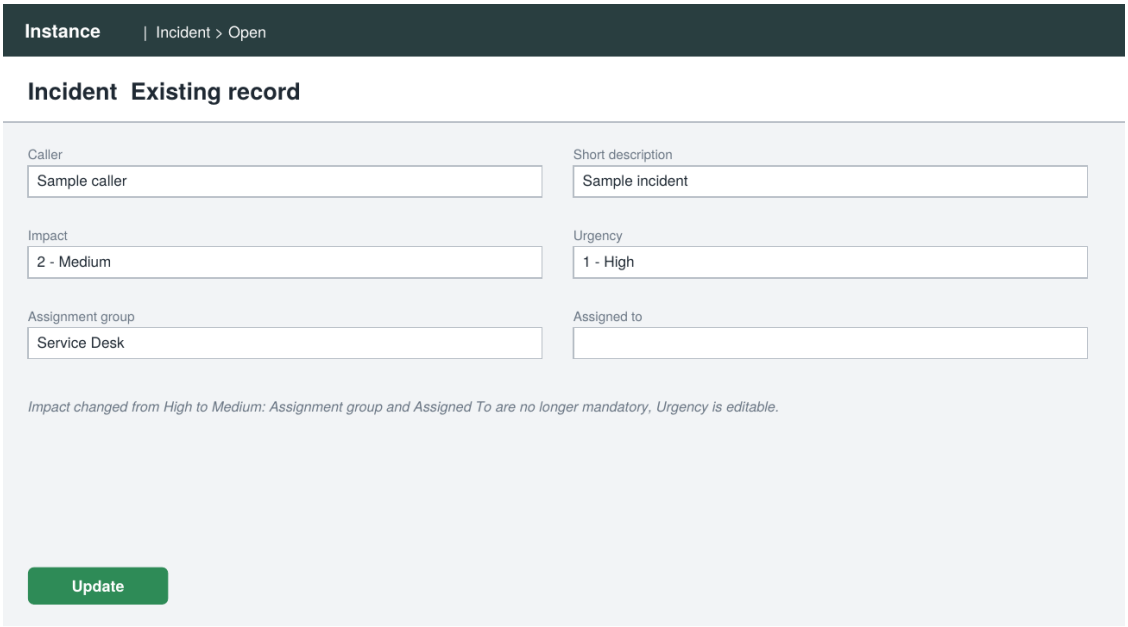


Figure 7.6: Reverse condition test: Impact changed from High to Medium (illustrative mock-up)

8. ADVANTAGES & DISADVANTAGES

Advantages

- Incidents are created with complete and valid information.
- Fields respond to Impact automatically, which saves effort and reduces mistakes.
- Reverse if false restores normal behaviour without extra work.
- Lightweight, easy to implement and easy to explain.
- Better reporting accuracy and SLA compliance because data is consistent.

Disadvantages

- The rules run only in the user interface, so data entered through other routes is not checked by these scripts.
- The scripts must be kept in step with the UI Policy when rules change.
- The browser alert on list edits can interrupt users.
- Administrative or configuration access is needed to set the controls up.

9. CONCLUSION

The Implement Client Script & UI Policy (Incident) micro project effectively demonstrates how UI Policies and Client Scripts can work together to enforce dynamic field behaviour, automate updates and prevent incorrect data submission on Incident forms. By using onChange, onSubmit and onCellEdit scripts alongside UI Policy actions, the project ensures that fields such as Assigned To and Urgency respond appropriately to changes in Impact, maintaining clean and consistent data.

This solution not only enforces mandatory fields and controls user interactions but also remains lightweight, efficient and easy to implement, making it ideal for short-duration micro projects while showcasing practical best practices for improving form usability and data integrity in ServiceNow.

10. FUTURE SCOPE

- Apply the same pattern to other fields and to other tables, such as Problem or Change.
- Add controls for other Impact or Priority levels.
- Add a server-side check as a backup, so the rules also apply when records are created outside the form.
- Show a message explaining why a field is read-only.
- Keep a reusable test checklist for every new rule.

11. APPENDIX

Source Code

onChange Client Script: Auto set urgency for high impact

```
function onChange(control, oldValue, newValue, isLoading) {
    if (isLoading || newValue == '') {
        return;
    }

    if (newValue == '1') {
        g_form.setValue('urgency', '1');
        g_form.addInfoMessage('Urgency set to High for High impact incident.');
    }
}
```

onSubmit Client Script: Prevent save if Assigned To missing

```
function onSubmit() {
    if (g_form.getValue('impact') == '1' &&
        g_form.getValue('assigned_to') == '') {

        g_form.showErrorBox(
            'assigned_to',
            'Assigned To is mandatory for High impact incidents.'
        );
        return false;
    }
    return true;
}
```

onCellEdit Client Script: Prevent state change via list edit

```
function onCellEdit(sysIDs, table, oldValues, newValue, callback) {

    alert('State cannot be updated using list editing. Please open the Incident.');

    callback(false);
}
```

Dataset Link

Not applicable. The project works on Incident records in the ServiceNow instance and uses no external dataset.